

LAPORAN PRAKTIKUM PEKAN 8



MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. WAHYUDI, S.T., M.T.

OLEH :

MUHAMMAD HANS NAFIS

NIM : 2511532027

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kepada Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 8 ini tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu tugas praktikum mata kuliah Basis Data, dengan topik pembahasan mengenai konsep dasar proses *sorting* (*Shell Sort*, *Quick Sort*, *Merge Sort*).

Penyusunan laporan ini bertujuan untuk mendokumentasikan hasil praktikum yang telah dilakukan, sekaligus sebagai saran untuk memperdalam pemahaman penulis terhadap materi yang telah dipelajari selama kegiatan praktikum berlangsung. Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengalaman dan pengetahuan baru, khususnya dalam memahami konsep dasar proses *sorting* serta penerapannya dalam pemograman menggunakan bahasa Java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, asisten dosen, teman-teman praktikum dan pihak lain yang turut mendukung penulisan laporan ini.

Selasa, 02 Juni 2026

Muhammad Hans Nafis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB 1	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB 2	2
PEMBAHASAN	2
2.1 <i>Sorting</i>	2
2.2 Class ShellSort.....	3
2.3 Class QuickSort.....	4
2.4 Class MergeSort	7
2.5 Class BubbleSortGUI	10
2.6 Class MergeSortGUI.....	18
2.7 Tugas Praktikum Pekan 8.....	27
BAB 3	38
KESIMPULAN.....	38
DAFTAR PUSTAKA	39
LAMPIRAN.....	40

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengolahan data, proses pengurutan data merupakan salah satu hal yang sangat penting. Data yang telah terurut akan mempermudah proses pencarian, pengelompokan, serta pengolahan informasi lainnya. Oleh karena itu, diperlukan pemahaman mengenai berbagai metode *sorting* agar programmer dapat memilih logaritma yang sesuai dengan kebutuhan.

Terdapat berbagai macam algoritma pengurutan data, diantaranya adalah *Shell Sort*, *Quick Sort*, dan *Merge Sort*. Ketiga metode tersebut memiliki cara kerja yang berbeda dalam mengurutkan data, namun sama-sama bertujuan menghasilkan data yang tersusun secara teratur, baik secara *ascending* maupun *descending*.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar algoritma pengurutan data (*sorting*).
2. Mengetahui cara kerja algoritma *Shell Sort*.
3. Mengetahui cara kerja algoritma *Quick Sort*.
4. Mengetahui cara kerja algoritma *Merge Sort*.

1.3 Manfaat Praktikum

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut :

1. Menambah pemahaman mengenai teknik pengurutan data.
2. Membantu memahami perbedaan proses kerja setiap algoritma *sorting*.
3. Meningkatkan kemampuan pemrograman dalam bahasa Java.
4. Melatih keterampilan dalam mengelola kode program secara terstruktur.

BAB 2

PEMBAHASAN

2.1 *Sorting*

Sorting merupakan proses mengurutkan data ke dalam urutan tertentu, seperti *ascending* atau *descending*. Pengurutan data sangat penting dalam pemrograman karena dapat mempermudah proses pencarian data, pengolahan data, serta meningkatkan efisiensi program. Terdapat berbagai macam algoritma pengurutan data, diantaranya adalah *Shell Sort*, *Quick Sort*, dan *Merge Sort*.

Shell Sort merupakan algoritma pengurutan yang dikembangkan oleh Donald Shell sebagai penyempurnaan dari *Insertion Sort*. Algoritma ini bekerja dengan membandingkan dan mengurutkan elemen-elemen yang berjauhan terlebih dahulu menggunakan nilai jarak (*gap*), kemudian secara bertahap mengurangi nilai *gap* hingga menjadi 1. Dengan cara tersebut, elemen yang berada jauh dari posisi seharusnya dapat dipindahkan lebih cepat dibandingkan *Insertion Sort* biasa. Proses pengurutan dilakukan berulang kali pada setiap nilai *gap* sampai seluruh data tersusun secara teratur. *Shell Sort* memiliki performa yang lebih baik daripada *Insertion Sort* pada data berukuran besar karena mampu mengurangi jumlah pergeseran elemen yang diperlukan selama proses pengurutan.

Quick Sort merupakan algoritma yang menggunakan metode *Divide and Conquer*. Algoritma ini bekerja dengan memilih sebuah elemen sebagai pivot, kemudian membagi data menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Setelah proses pembagian selesai, *Quick Sort* akan secara rekursif mengurutkan kedua bagian tersebut hingga seluruh data tersusun dengan benar. Keunggulan *Quick Sort* terletak pada efisiensinya yang tinggi dengan kompleksitas rata-rata sebesar $O(n \log n)$, sehingga sering digunakan dalam berbagai aplikasi pengolahan data. Namun, jika pemilihan pivot kurang tepat, performanya dapat menurun hingga $O(n^2)$.

Merge Sort adalah algoritma pengurutan yang juga menerapkan pendekatan *Divide and Conquer*. Algoritma ini bekerja dengan membagi data menjadi dua bagian yang lebih kecil secara berulang hingga setiap bagian hanya berisi satu elemen. Setelah itu, bagian-bagian tersebut digabungkan kembali sambil diurutkan sehingga menghasilkan data yang tersusun secara teratur. Salah satu kelebihan

Merge Sort adalah performanya yang stabil dengan kompleksitas waktu $O(n \log n)$ pada kondisi terbaik, rata-rata, maupun terburuk. Selain itu, algoritma ini mampu mempertahankan urutan relatif data yang memiliki nilai sama. Namun, *Merge Sort* memerlukan ruang memori tambahan untuk proses penggabungan data, sehingga penggunaan memorinya lebih besar dibandingkan beberapa algoritma pengurutan lainnya.

2.2 Class ShellSort

Berikut kode program Java mengenai class ShellSort :

```
public class ShellSort_2511532027 {
    public static void shellSort_2027(int[] A_2027) {
        int n_2027 = A_2027.length;
        int gap_2027 = n_2027 / 2;
        while (gap_2027 > 0) {
            for (int i_2027 = gap_2027; i_2027 < n_2027;
i_2027++) {
                int temp_2027 = A_2027[i_2027];
                int j_2027 = i_2027;
                while (j_2027 >= gap_2027 && A_2027[j_2027 -
gap_2027] > temp_2027) {
                    A_2027[j_2027] = A_2027[j_2027 -
gap_2027];
                    j_2027 = j_2027 - gap_2027;
                }
                A_2027[j_2027] = temp_2027;
            }
            gap_2027 = gap_2027 / 2;
        }
    }

    public static void main (String[] args) {
        int[] data_2027 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
        System.out.print("Sebelum: ");
        printArray_2027(data_2027);
        shellSort_2027(data_2027);
        System.out.print("Sesudah (Shell Sort): ");
        printArray_2027(data_2027);
    }

    public static void printArray_2027(int[] arr_2027) {
```

```

        for (int i_2027 : arr_2027) System.out.print(i_2027 + " ");
        System.out.println();
    }
}

```

Kode program tersebut digunakan untuk mengurutkan data bertipe integer menggunakan algoritma *Shell Sort*. Pada program ini, proses pengurutan dilakukan dalam method `shellSort_2027(int[] A_2027)` yang menerima sebuah *array* sebagai parameter. Nilai *gap* awal ditentukan sebesar setengah dari jumlah elemen *array*, kemudian secara bertahap diperkecil hingga bernilai 0. Selama proses tersebut, elemen-elemen *array* dibandingkan dan dipindahkan ke posisi yang sesuai sehingga data menjadi semakin terurut pada setiap iterasi.

Di dalam method `shellSort_2027()`, terdapat perulangan `while` yang mengatur nilai *gap* dan perulangan `for` yang digunakan untuk menelusuri elemen-elemen *array*. Variabel `temp_2027` berfungsi sebagai tempat penyimpanan sementara nilai yang sedang diproses, sedangkan variabel `j_2027` digunakan untuk menentukan posisi perpindahan elemen selama proses penggeseran. Apabila elemen yang berada pada jarak *gap* sebelumnya lebih besar daripada nilai yang disimpan pada `temp_2027`, maka elemen tersebut akan digeser ke kanan hingga ditemukan posisi yang tepat. Setelah seluruh proses perbandingan dan penggeseran selesai, nilai `temp_2027` ditempatkan pada posisi yang sesuai sehingga urutan data menjadi lebih teratur.

Method `main()` berfungsi sebagai program utama yang menyediakan data awal berupa *array* {3, 10, 4, 6, 8, 9, 7, 2, 1, 5}. Sebelum dilakukan pengurutan, isi *array* ditampilkan menggunakan method `printArray_2027()`. Selanjutnya method `shellSort_2027()` dipanggil untuk mengurutkan data secara *ascending*, dan hasil pengurutan kembali ditampilkan ke layar. Method `printArray_2027()` sendiri digunakan untuk mencetak seluruh elemen *array* ke konsol sehingga pengguna dapat melihat perbedaan kondisi data sebelum dan sesudah proses pengurutan dilakukan.

2.3 Class QuickSort

Berikut kode program Java mengenai class `QuickSort` :

```

public class QuickSort_2511532027 {
    static void swap_2027(int[] arr_2027, int i_2027, int j_2027)

```

```

    {
        int temp_2027 = arr_2027[i_2027];
        arr_2027[i_2027] = arr_2027[j_2027];
        arr_2027[j_2027] = temp_2027;
    }

    // Metode tambahan untuk mengatur pivot menggunakan Median-of-
Three
    static void medianOfThree_2027(int[] arr_2027, int low_2027, int
high_2027)
    {
        int mid_2027 = low_2027 + (high_2027 - low_2027) / 2;
        // Urutkan elemen low, mid, dan high
        if (arr_2027[low_2027] > arr_2027[mid_2027]) {
            swap_2027(arr_2027, low_2027, mid_2027);
        }
        if (arr_2027[low_2027] > arr_2027[high_2027]) {
            swap_2027(arr_2027, low_2027, high_2027);
        }
        if (arr_2027[mid_2027] > arr_2027[high_2027]) {
            swap_2027(arr_2027, mid_2027, high_2027);
        }
        swap_2027(arr_2027, mid_2027, high_2027);
    }

    static int partition_2027(int[] arr_2027, int low_2027, int
high_2027)
    {
        // Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_2027(arr_2027, low_2027, high_2027);
        int pivot_2027 = arr_2027[high_2027]; // Sekarang
arr_2027[high_2027] sudah berisi nilai median
        int i_2027 = (low_2027 - 1);
        for (int j_2027 = low_2027; j_2027 <= high_2027 - 1;
j_2027++) {
            // Jika elemen saat ini lebih kecil dari atau sama
dengan pivot
            if (arr_2027[j_2027] < pivot_2027) {
                // Increment indeks elemen yang lebih kecil
                i_2027++;
            }
        }
    }

```



```

        swap_2027(arr_2027, i_2027, j_2027);
    }
}
swap_2027(arr_2027, i_2027 + 1, high_2027);
return (i_2027 + 1);
}
static void quickSort_2027(int[] arr_2027, int low_2027, int
high_2027)
{
    if (low_2027 < high_2027) {
        int pi_2027 = partition_2027(arr_2027, low_2027,
high_2027);
        quickSort_2027(arr_2027, low_2027, pi_2027 - 1);
        quickSort_2027(arr_2027, pi_2027 + 1, high_2027);
    }
}
public static void printArr_2027(int[] arr_2027)
{
    for (int i_2027 = 0; i_2027 < arr_2027.length; i_2027++) {
        System.out.print(arr_2027[i_2027] + " ");
    }
    System.out.println();
}
public static void main(String[] args)
{
    int[] arr_2027 = {10, 7, 8, 9, 1, 5};
    int N_2027 = arr_2027.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_2027(arr_2027);
    quickSort_2027(arr_2027, 0, N_2027 - 1);
    System.out.print("Data Terurut quicksort: ");
    printArr_2027(arr_2027);
}
}

```

Kode program tersebut digunakan untuk mengurutkan data bertipe integer menggunakan algoritma *Quick Sort* dengan teknik pemilihan pivot *Median of Three*. Program ini memiliki method `swap_2027()` yang berfungsi untuk menukar

posisi dua elemen dalam *array*. Method tersebut digunakan pada berbagai tahap proses pengurutan, termasuk saat menentukan pivot dan melakukan partisi data. Selain itu, terdapat method `medianOfThree_2027()` yang digunakan untuk memilih pivot dengan cara membandingkan elemen pada indeks awal, tengah, dan akhir. Setelah ketiga elemen tersebut diurutkan, nilai median dipindahkan ke posisi terakhir *array* sehingga dapat digunakan sebagai pivot pada proses partisi.

Proses pembagian data dilakukan pada method `partition_2027()`. Method ini terlebih dahulu memanggil `medianOfThree_2027()` untuk menentukan pivot, kemudian membandingkan setiap elemen dengan nilai pivot. Apabila ditemukan elemen yang lebih kecil dari pivot, elemen tersebut akan dipindahkan ke bagian kiri *array* menggunakan method `swap_2027()`. Setelah seluruh elemen selesai dibandingkan, pivot ditempatkan pada posisi yang tepat sehingga elemen di sebelah kiri bernilai lebih kecil dan elemen di sebelah kanan bernilai lebih besar. Method ini kemudian mengembalikan indeks posisi pivot yang digunakan untuk membagi *array* menjadi dua bagian.

Pengurutan utama dilakukan oleh method `quickSort_2027()` yang bekerja secara rekursif. Setelah mendapatkan posisi pivot dari method `partition_2027()`, method ini akan memanggil dirinya sendiri untuk mengurutkan bagian kiri dan bagian kanan pivot hingga seluruh elemen *array* berada pada urutan yang benar. Dengan pendekatan rekursif tersebut, ukuran *sub-array* akan terus mengecil sampai hanya tersisa satu elemen atau tidak ada elemen yang perlu diurutkan lagi.

Pada method `main()`, program mendefinisikan data awal berupa *array* {10, 7, 8, 9, 1, 5}. Data tersebut ditampilkan terlebih dahulu menggunakan method `printArr_2027()`, kemudian diurutkan dengan memanggil method `quickSort_2027()`. Setelah proses pengurutan selesai, hasil akhir ditampilkan ke layar.

2.4 Class MergeSort

Berikut kode program Java mengenai class MergeSort :

```
public class MergeSort_2511532027 {  
    void merge_2027(int arr_2027[], int l_2027, int m_2027, int  
r_2027) {  
        // Find sizes of two subarrays to be merged  
        int n1_2027 = m_2027 - l_2027 + 1;
```

```

int n2_2027 = r_2027 - m_2027;
/* Create temp arrays */
int L_2027[] = new int[n1_2027];
int R_2027[] = new int[n2_2027];
/* Copy data to temp arrays */
for (int i_2027 = 0; i_2027 < n1_2027; ++i_2027)
    L_2027[i_2027] = arr_2027[l_2027 + i_2027];
for (int j_2027 = 0; j_2027 < n2_2027; ++j_2027)
    R_2027[j_2027] = arr_2027[m_2027 + 1 + j_2027];
int i_2027 = 0, j_2027 = 0;
// Initial index of merged subarray array
int k_2027 = l_2027;
while (i_2027 < n1_2027 && j_2027 < n2_2027) {
    if (L_2027[i_2027] <= R_2027[j_2027]) {
        arr_2027[k_2027] = L_2027[i_2027];
        i_2027++;
    } else {
        arr_2027[k_2027] = R_2027[j_2027];
        j_2027++;
    }
    k_2027++;
}
/* Copy remaining elements of L_2027[] if any */
while (i_2027 < n1_2027) {
    arr_2027[k_2027] = L_2027[i_2027];
    i_2027++;
    k_2027++;
}
/* Copy remaining elements of R_2027[] if any */
while (j_2027 < n2_2027) {
    arr_2027[k_2027] = R_2027[j_2027];
    j_2027++;
    k_2027++;
}
}

void sort_2027(int arr_2027[], int l_2027, int r_2027) {
    if (l_2027 < r_2027) {
        // Find the middle point

```

```

        int m_2027 = (l_2027 + r_2027) / 2;
        // Sort first and second halves
        sort_2027(arr_2027, l_2027, m_2027);
        sort_2027(arr_2027, m_2027 + 1, r_2027);
        // Merge the sorted halves
        merge_2027(arr_2027, l_2027, m_2027, r_2027);
    }
}

/* A utility function to print array of size n */
static void printArray_2027(int arr_2027[]) {
    int n_2027 = arr_2027.length;
    for (int i_2027 = 0; i_2027 < n_2027; ++i_2027)
        System.out.print(arr_2027[i_2027] + " ");
    System.out.println();
}

public static void main (String[] args) {
    int arr_2027[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut");
    printArray_2027(arr_2027);
    MergeSort_2511532027 ob_2027 = new MergeSort_2511532027();
    ob_2027.sort_2027(arr_2027, 0, arr_2027.length - 1);
    System.out.println("\nSesudah Terurut menggunakan merge
Sort");

    printArray_2027(arr_2027);
}
}

```

Kode program tersebut digunakan untuk mengurutkan tipe data bertipe integer menggunakan algoritma *Merge Sort*. Proses utama pengurutan dilakukan pada method `sort_2027()`. Method ini bekerja secara rekursif dengan membagi *array* menjadi dua bagian hingga setiap bagian hanya terdiri dari satu elemen. Nilai tengah *array* ditentukan menggunakan variabel `m_2027`, kemudian method `sort_2027()` dipanggil kembali untuk mengurutkan bagian kiri dan bagian kanan *array*. Setelah kedua bagian diurutkan, method `merge_2027()` digunakan untuk menggabungkan kedua *sub-array* tersebut menjadi satu *array* yang telah tersusun secara berurutan.

Method `merge_2027()` berfungsi untuk menggabungkan dua *sub-array* yang telah terurut. Pada method ini dibuat dua *array* sementara, yaitu `L_2027` dan `R_2027`, yang digunakan untuk menyimpan elemen dari bagian kiri dan bagian kanan *array* asli. Selanjutnya dilakukan proses perbandingan antara elemen pada kedua *array* sementara tersebut. Elemen yang memiliki nilai lebih kecil akan ditempatkan terlebih dahulu ke dalam *array* utama. Proses ini dilakukan berulang hingga seluruh elemen dari kedua *sub-array* berhasil digabungkan. Jika masih terdapat elemen yang tersisa pada salah satu *array* sementara, elemen tersebut akan langsung disalin ke *array* utama sehingga tidak ada data yang hilang.

Selain itu, program menyediakan method `printArray_2027()` yang digunakan untuk menampilkan isis *array* ke layar. Method ini melakukan perulangan terhadap seluruh elemen *array* dan mencetaknya secara berurutan sehingga pengguna dapat melihat kondisi data sebelum maupun sesudah proses pengurutan dilakukan.

Pada method `main()`, program mendefinisikan *array* berisi data {12, 11, 13, 5, 6, 7} yang akan diurutkan. Data awal ditampilkan terlebih dahulu menggunakan method `printArray_2027()`. Selanjutnya dibuat objek `MergeSort_2511532027` dan method `sort_2027()` dipanggil untuk melakukan proses pengurutan. Setelah seluruh proses selesai, hasil akhir ditampilkan kembali ke layar.

2.5 Class BubbleSortGUI

Berikut kode program Java mengenai class `BubbleSortGUI` :

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
```

```

import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class BubbleSortGUI_2511532027 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array_2027;
    private JLabel[] labelArray_2027;
    private JButton stepButton_2027, resetButton_2027,
setButton_2027;
    private JTextField inputField_2027;
    private JPanel panelArray_2027;
    private JTextArea stepArea_2027;
    private int i_2027 = 1, j_2027;
    private boolean sorting_2027 = false;
    private int stepCount_2027 = 1;

    /**
     * Create the frame.
     */
    public BubbleSortGUI_2511532027() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        //Panel input
        JPanel inputPanel_2027 = new JPanel (new FlowLayout());
        inputField_2027 = new JTextField(30);
        setButton_2027 = new JButton("Set Array");
        inputPanel_2027.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
        inputPanel_2027.add(inputField_2027);
        inputPanel_2027.add(setButton_2027);

        // Panel array visual
        panelArray_2027 = new JPanel();

```

```

        panelArray_2027.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_2027 = new JPanel();
        stepButton_2027 = new JButton("Langkah Selanjutnya");
        resetButton_2027 = new JButton("Reset");
        stepButton_2027.setEnabled(false);
        controlPanel_2027.add(stepButton_2027);
        controlPanel_2027.add(resetButton_2027);

        // Area teks untuk log langkah-langkah
        stepArea_2027 = new JTextArea(8, 60);
        stepArea_2027.setEditable(false);
        stepArea_2027.setFont(new Font("Monospaced", Font.PLAIN,
14));

        JScrollPane scrollPane_2027 = new
JScrollPane(stepArea_2027);

        // Tambahkan panel ke frame
        add(inputPanel_2027, BorderLayout.NORTH);
        add(panelArray_2027, BorderLayout.CENTER);
        add(controlPanel_2027, BorderLayout.SOUTH);
        add(scrollPane_2027, BorderLayout.EAST);

        // Event Set Array
        setButton_2027.addActionListener(e_2027 ->
setArrayFromInput_2027());

        // Event Langkah Selanjutnya
        stepButton_2027.addActionListener(e_2027 ->
performStep_2027());

        // Event Reset
        resetButton_2027.addActionListener(e_2027 ->
reset_2027());}

        private void setArrayFromInput_2027() {
            String text_2027 = inputField_2027.getText().trim();

```

```

        if (text_2027.isEmpty()) return;
        String[] parts_2027 = text_2027.split(",");
        array_2027 = new int[parts_2027.length];
        try {
            for (int k_2027 = 0; k_2027 <
parts_2027.length; k_2027++) {
                array_2027[k_2027] =
Integer.parseInt(parts_2027[k_2027].trim());
            } catch (NumberFormatException e_2027) {
                JOptionPane.showMessageDialog(this,
"Masukkan hanya angka "
                + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
                return; }
            i_2027 = 0;
            j_2027 = 0;
            stepCount_2027 = 1;
            sorting_2027 = true;
            stepButton_2027.setEnabled(true);
            stepArea_2027.setText("");
            panelArray_2027.removeAll();
            labelArray_2027 = new
JLabel[array_2027.length];
            for (int k_2027 = 0; k_2027 <
array_2027.length; k_2027++) {
                labelArray_2027[k_2027] = new
JLabel(String.valueOf(array_2027[k_2027]));
                labelArray_2027[k_2027].setFont(new
Font("Arial", Font.BOLD, 24));

                labelArray_2027[k_2027].setOpaque(true);

                labelArray_2027[k_2027].setBackground(Color.WHITE);

                labelArray_2027[k_2027].setBorder(BorderFactory.createLineBorder(
Color.BLACK));

                labelArray_2027[k_2027].setPreferredSize(new Dimension(50, 50));

```



```

        labelArray_2027[k_2027].setHorizontalAlignment(SwingConstants.CEN
TER);

        panelArray_2027.add(labelArray_2027[k_2027]);
    }
    panelArray_2027.revalidate();
    panelArray_2027.repaint();
}

private void performStep_2027() {
    if (!sorting_2027 || i_2027 >=
array_2027.length - 1) {
        sorting_2027 = false;
        stepButton_2027.setEnabled(false);
        JOptionPane.showMessageDialog(this,
"Sorting selesai!");

        return; }
    resetHighlights_2027();
    StringBuilder stepLog_2027 = new
StringBuilder();

    labelArray_2027[j_2027].setBackground(Color.CYAN);
    labelArray_2027[j_2027 +
1].setBackground(Color.CYAN);

    if (array_2027[j_2027] >
array_2027[j_2027 + 1]) {
        // swap
        int temp_2027 = array_2027[j_2027];
        array_2027[j_2027] = array_2027[j_2027
+ 1];

        array_2027[j_2027 + 1] = temp_2027;

        labelArray_2027[j_2027].setBackground(Color.RED);
        labelArray_2027[j_2027 +
1].setBackground(Color.RED);

        stepLog_2027.append("Langkah
").append(stepCount_2027).append(": Menukar elemen ke-")

```

```

        .append(j_2027).append("
(").append(array_2027[j_2027 + 1]).append(" dengan ke-")
        .append(j_2027 + 1).append("
(").append(array_2027[j_2027]).append(")\n");
    } else {
        stepLog_2027.append("Langkah
").append(stepCount_2027).append(": Tidak ada pertukaran antara ke-")
        .append(j_2027).append(" dan ke-
").append(j_2027 + 1).append(")\n");
        stepLog_2027.append("Hasil:
").append(arrayToString_2027(array_2027)).append(")\n\n");

        stepArea_2027.append(stepLog_2027.toString());
        updateLabels_2027();
        j_2027++;
        if (j_2027 >= array_2027.length - i_2027 - 1)
{
            j_2027 = 0;
            i_2027++;
        }
        stepCount_2027++;
        if (i_2027 >= array_2027.length - 1) {
            sorting_2027 = false;
            stepButton_2027.setEnabled(false);
            JOptionPane.showMessageDialog(this,
"Sorting selesai!");
        }
    }
}

private void updateLabels_2027() {
    for (int k_2027 = 0; k_2027 <
array_2027.length; k_2027++) {

        labelArray_2027[k_2027].setText(String.valueOf(array_2027[k_2027]
));
    }
}
}

```

```

        private void resetHighlights_2027() {
            for (JLabel label : labelArray_2027) {
                label.setBackground(Color.WHITE);
            }
        }

        private void reset_2027() {
            inputField_2027.setText("");
            panelArray_2027.removeAll();
            panelArray_2027.revalidate();
            panelArray_2027.repaint();
            stepArea_2027.setText("");
            stepButton_2027.setEnabled(false);
            sorting_2027 = false;
            i_2027 = 0;
            j_2027 = 0;
            stepCount_2027 = 1;
        }

        private String arrayToString_2027(int[] arr_2027) {
            StringBuilder sb_2027 = new StringBuilder();
            for (int k_2027 = 0; k_2027 <
arr_2027.length; k_2027++) {
                sb_2027.append(arr_2027[k_2027]);
                if (k_2027 < arr_2027.length - 1)
sb_2027.append(", ");
            }
            return sb_2027.toString();
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> {
                BubbleSortGUI_2511532027 gui_2027 = new
BubbleSortGUI_2511532027();

                gui_2027.setVisible(true);
            });
        }
    }
}

```

Kode program tersebut digunakan untuk mengimplementasikan algoritma *Bubble Sort* dalam bentuk aplikasi GUI menggunakan pustaka Java Swing. Program ini memungkinkan pengguna memasukkan sejumlah data angka melalui kotak input, kemudian mengamati proses pengurutan data secara bertahap melalui tombol “Langkah Selanjutnya”. Selain menampilkan hasil pengurutan secara visual, program juga memberikan informasi setiap langkah yang terjadi selama proses *Bubble Sort* sehingga pengguna dapat memahami cara kerja algoritma secara lebih mudah dan interaktif.

Kelas `BubbleSortGUI_2511532027` merupakan kelas utama yang mewarisi kelas `JFrame` sehingga dapat berfungsi sebagai jendela aplikasi. Pada konstruktor `BubbleSortGUI_2511532027()`, program membangun tampilan antarmuka yang terdiri dari panel input, panel visualisasi *array*, panel kontrol, dan area log langkah-langkah proses *sorting*. Pengguna dapat memasukkan data angka yang dipisahkan dengan tanda koma pada komponen `inputField_2027()`, kemudian menekan tombol `Set Array` untuk menampilkan data tersebut dalam bentuk label-label pada panel visualisasi. Tombol `Langkah Selanjutnya` digunakan untuk menjalankan satu langkah proses *Bubble Sort*, sedangkan tombol `Resest` digunakan untuk mengembalikan program ke kondisi awal.

Method `setArrayFromInput_2027()` berfungsi untuk membaca data yang dimasukkan pengguna, mengubahnya menjadi *array* bertipe integer, serta menampilkan setiap elemen *array* ke dalam panel visualisasi. Program juga melakukan validasi input untuk memastikan bahwa data yang dimasukkan berupa angka. Setelah data berhasil dimasukkan, variabel-variabel pengendali seperti `i_2027`, `j_2027`, dan `stepCount_2027` akan diinisialisasi ulang agar proses pengurutan dapat dimulai dari awal.

Proses utama *Bubble Sort* dilakukan pada method `performStep_2027()`. Method ini menjalankan satu langkah perbandingan setiap kali tombol `Langkah Selanjutnya` ditekan. Program akan membandingkan dua elemen yang berdekatan, yaitu elemen pada indeks `j_2027` dan `j_2027 + 1`. Jika elemen pertama lebih besar daripada elemen kedua, maka kedua elemen tersebut akan ditukar posisinya. Selama proses berlangsung, elemen yang sedang dibandingkan akan diberi warna biru, sedangkan elemen yang mengalami pertukaran akan diberi warna merah.

Setiap aktivitas yang terjadi akan dicatat ke dalam `stepArea_2027` sehingga pengguna dapat melihat urutan langkah-langkah yang dilakukan oleh algoritma.

Method `updateLabels_2027()` digunakan untuk memperbarui tampilan label setelah terjadi perubahan posisi data, sedangkan method `resetHighlights_2027()` berfungsi mengembalikan warna seluruh label menjadi putih sebelum langkah berikutnya dijalankan. Selain itu, method `arrayToString_2027()` digunakan untuk mengubah isi *array* menjadi bentuk teks agar dapat ditampilkan pada area log. Method `reset_2027()` berfungsi menghapus seluruh data, membersihkan tampilan visualisasi, mengosongkan area log, serta mengembalikan seluruh variabel ke kondisi awal sehingga pengguna dapat melakukan percobaan baru.

Pada method `main()`, program dijalankan menggunakan `SwingUtilities.invokeLater()` untuk memastikan seluruh komponen GUI dibuat dan dijalankan pada *Event Dispatch Thread* (EDT) sesuai standar pemrograman Java Swing. Setelah objek `BubbleSortGUI_2511532027` dibuat dan ditampilkan, pengguna dapat memasukkan data dan mengamati proses pengurutan *Bubble Sort* langkah demi langkah.

2.6 Class MergeSortGUI

Berikut kode program Java mengenai class `MergeSortGUI` :

```
import java.util.LinkedList;
import java.util.Queue;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
```

```

import javax.swing.SwingUtilities;

public class MergeSortGUI_2511532027 extends JFrame {
    private static final long serialVersionUID = 1L;
    private Queue<int[]> mergeQueue_2027 = new LinkedList<>();
    private int[] temp_2027;
    private int[] array_2027;
    private JLabel[] labelArray_2027;
    private JButton stepButton_2027, resetButton_2027,
setButton_2027;
    private JTextField inputField_2027;
    private JPanel panelArray_2027;
    private JTextArea stepArea_2027;
    private int left_2027, mid_2027, right_2027;
    private int i_2027 = 1, j_2027, k_2027;
    private boolean copying_2027 = false;
    private boolean isMerging_2027 = false;
    private int stepCount_2027 = 1;

    /**
     * Create the frame.
     */
    public MergeSortGUI_2511532027() {
        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        //Panel input
        JPanel inputPanel_2027 = new JPanel (new FlowLayout());
        inputField_2027 = new JTextField(30);
        setButton_2027 = new JButton("Set Array");
        inputPanel_2027.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
        inputPanel_2027.add(inputField_2027);
        inputPanel_2027.add(setButton_2027);

```

```

// Panel array visual
panelArray_2027 = new JPanel();
panelArray_2027.setLayout(new FlowLayout());

// Panel kontrol
JPanel controlPanel_2027 = new JPanel();
stepButton_2027 = new JButton("Langkah Selanjutnya");
resetButton_2027 = new JButton("Reset");
stepButton_2027.setEnabled(false);
controlPanel_2027.add(stepButton_2027);
controlPanel_2027.add(resetButton_2027);

// Area teks untuk log langkah-langkah
stepArea_2027 = new JTextArea(8, 60);
stepArea_2027.setEditable(false);
stepArea_2027.setFont(new Font("Monospaced", Font.PLAIN,
14));

JScrollPane scrollPane_2027 = new
JScrollPane(stepArea_2027);

// Tambahkan panel ke frame
add(inputPanel_2027, BorderLayout.NORTH);
add(panelArray_2027, BorderLayout.CENTER);
add(controlPanel_2027, BorderLayout.SOUTH);
add(scrollPane_2027, BorderLayout.EAST);

// Event Set Array
setButton_2027.addActionListener(e_2027 ->
setArrayFromInput_2027());

// Event Langkah Selanjutnya
stepButton_2027.addActionListener(e_2027 ->
performStep_2027());

// Event Reset
resetButton_2027.addActionListener(e_2027 ->
reset_2027());}

```

```

private void setArrayFromInput_2027() {
    String text_2027 = inputField_2027.getText().trim();
    if (text_2027.isEmpty()) return;
    String[] parts_2027 = text_2027.split(",");
    array_2027 = new int[parts_2027.length];
    try {
        for (int i_2027 = 0; i_2027 <
parts_2027.length; i_2027++) {
            array_2027[i_2027] =
Integer.parseInt(parts_2027[i_2027].trim());
        } catch (NumberFormatException e_2027) {
            JOptionPane.showMessageDialog(this,
"Masukkan hanya angka!",
            "Error", JOptionPane.ERROR_MESSAGE);
            return; }
        labelArray_2027 = new
JLabel[array_2027.length];
        panelArray_2027.removeAll();
        for (int i_2027 = 0; i_2027 <
array_2027.length; i_2027++) {
            labelArray_2027[i_2027] = new
JLabel(String.valueOf(array_2027[i_2027]));
            labelArray_2027[i_2027].setFont(new
Font("Arial", Font.BOLD, 24));

            labelArray_2027[i_2027].setOpaque(true);

            labelArray_2027[i_2027].setBackground(Color.WHITE);

            labelArray_2027[i_2027].setBorder(BorderFactory.createLineBorder(
Color.BLACK));

            labelArray_2027[i_2027].setPreferredSize(new Dimension(50, 50));

            labelArray_2027[i_2027].setHorizontalAlignment(SwingConstants.CEN
TER);

            panelArray_2027.add(labelArray_2027[i_2027]);

```



```

    }
    mergeQueue_2027.clear();
    generateMergeSteps_2027(0, array_2027.length
- 1);

    stepButton_2027.setEnabled(true);
    stepArea_2027.setText("");
    stepCount_2027 = 1;
    isMerging_2027 = false;
    panelArray_2027.revalidate();
    panelArray_2027.repaint();
}

private void performStep_2027() {
    resetHighlights_2027();
    if (!isMerging_2027 &&
!mergeQueue_2027.isEmpty()) {
        int[] range_2027 =
mergeQueue_2027.poll();

        left_2027 = range_2027[0];
        mid_2027 = range_2027[1];
        right_2027 = range_2027[2];
        temp_2027 = new int[right_2027 -
left_2027 + 1];

        i_2027 = left_2027;
        j_2027 = mid_2027 + 1;
        k_2027 = 0;
        copying_2027 = false;
        isMerging_2027 = false;
        stepArea_2027.append("Langkah " +
stepCount_2027++ +
                                ": Mulai merge dari " +
left_2027 + " ke " + right_2027 + "\n");
        return; }
    if (isMerging_2027 && !copying_2027) {
        if (i_2027 <= mid_2027 && j_2027
<= right_2027) {

            labelArray_2027[i_2027].setBackground(Color.CYAN);

```

```

        labelArray_2027[j_2027].setBackground(Color.CYAN);
        if (array_2027[i_2027] <=
array_2027[j_2027]) {
            temp_2027[k_2027++]
= array_2027[i_2027++];
        } else {
            temp_2027[k_2027++]
= array_2027[j_2027++];
        }

        stepArea_2027.append("Langkah " + stepCount_2027++ + ":
Bandingkan dan salin elemen\n");

        return;
    } else if (i_2027 <= mid_2027) {
        temp_2027[k_2027++] =
array_2027[i_2027++];

        stepArea_2027.append("Langkah " + stepCount_2027++ + ": Salin
sis kiri\n");

        return;
    } else if (j_2027 <= right_2027)
{
        temp_2027[k_2027++] =
array_2027[j_2027++];

        stepArea_2027.append("Langkah " + stepCount_2027++ + ": Salin
sis kanan\n");

        return;
    } else {
        copying_2027 = true;
        k_2027 = 0;
        return;
    }
}

if (copying_2027 && k_2027 <
temp_2027.length) {

```

```

array_2027[left_2027 + k_2027] =
temp_2027[k_2027];

labelArray_2027[left_2027 +
k_2027].setText(String.valueOf(temp_2027[k_2027]));
labelArray_2027[left_2027 +
k_2027].setBackground(Color.GREEN);

k_2027++;
stepArea_2027.append("Langkah "
+ stepCount_2027++ + ": Tempelkan ke array utama\n");

return;
}

if (copying_2027 && k_2027 ==
temp_2027.length) {

    isMerging_2027 = false;
    copying_2027 = false;
}

if (mergeQueue_2027.isEmpty() &&
!isMerging_2027) {

    stepArea_2027.append("Selesai.\n");

    stepButton_2027.setEnabled(false);

    JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
}

}

private void resetHighlights_2027() {
    if (labelArray_2027 == null) return;
    for (JLabel label : labelArray_2027) {
        label.setBackground(Color.WHITE);
    }
}

private void reset_2027() {
    inputField_2027.setText("");
    panelArray_2027.removeAll();
    panelArray_2027.revalidate();
    panelArray_2027.repaint();
    stepArea_2027.setText("");
}

```

```

        stepButton_2027.setEnabled(false);
        mergeQueue_2027.clear();
        isMerging_2027 = false;
        stepCount_2027 = 1;
    }
    private void generateMergeSteps_2027(int left_2027,
int right_2027) {
        if (left_2027 < right_2027) {
            int mid_2027 = (left_2027 + right_2027)
/ 2;

            generateMergeSteps_2027(left_2027,
mid_2027);

            generateMergeSteps_2027(mid_2027 + 1,
right_2027);

            mergeQueue_2027.add(new int[]
{left_2027, mid_2027, right_2027});
        }
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            MergeSortGUI_2511532027 gui_2027 = new
MergeSortGUI_2511532027();

            gui_2027.setVisible(true);

        });
    }
}

```

Kode program tersebut digunakan untuk mengimplementasikan algoritma *Merge Sort* dalam bentuk aplikasi GUI menggunakan Java Swing. Program ini memungkinkan pengguna memasukkan sejumlah data angka melalui kotak input, kemudian mengamati proses pengurutan *Merge Sort* secara bertahap dan menekan tombol “Langkah Selanjutnya”. Selain menampilkan perubahan susunan data secara visual, program juga mencatat setiap langkah proses penggabungan sehingga pengguna dapat memahami cara kerja algoritma *Merge Sort* secara lebih jelas dan interaktif.

Kelas `MergeSortGUI_2511532027` merupakan kelas utama yang mewarisi kelas `JFrame` untuk membangun tampilan aplikasi. Pada

konstruktorMergeSortGUI_2511532027(), program membuat berbagai komponen antarmuka seperti panel input, panel visualisasi *array*, panel kontrol, dan area log proses pengurutan. Komponen `inputField_2027` digunakan untuk memasukkan data angka yang dipisahkan dengan tanda koma, sedangkan tombol Set Array berfungsi untuk memproses dan menampilkan data tersebut ke dalam panel visualisasi. Tombol Langkah Selanjutnya digunakan untuk menjalankan satu tahap proses *Merge Sort*, sedangkan tombol Reset digunakan untuk menghapus data dan mengembalikan program ke kondisi awal.

Method `setArrayFromInput_2027()` bertugas membaca data yang dimasukkan pengguna, mengubahnya menjadi *array* bertipe integer, dan menampilkan setiap elemen *array* dalam bentuk label pada panel visualisasi. Program juga melakukan validasi untuk memastikan bahwa seluruh input berupa angka. Setelah data berhasil dimasukkan, method `generateMergeSteps_2027()` dipanggil untuk menghasilkan seluruh tahapan penggabungan yang akan dilakukan oleh algoritma *Merge Sort*. Tahapan-tahapan tersebut disimpan ke dalam struktur data `Queue<int[]>` bernama `mergeQueue_2027` sehingga proses pengurutan dapat ditampilkan secara berurutan sesuai langkah-langkah *Merge Sort*.

Proses utama visualisasi dilakukan pada method `performStep_2027()`. Method ini mengatur setiap langkah penggabungan data yang telah dibagi sebelumnya. Ketika sebuah rentang data diambil dari `mergeQueue_2027`, program menentukan batas kiri, tengah, dan kanan yang akan digabungkan. Selanjutnya elemen dari kedua bagian dibandingkan satu per satu dan disalin ke dalam *array* sementara `temp_2027`. Selama proses berlangsung, elemen yang sedang dibandingkan akan diberi warna biru untuk menunjukkan bagian yang sedang diproses.

Setelah seluruh elemen dari kedua *sub-array* berhasil dibandingkan dan disimpan ke dalam *array* sementara, program memasuki tahap penyalinan kembali ke *array* utama. Pada tahap ini, setiap elemen yang ditempatkan kembali ke posisi yang sesuai akan ditampilkan dengan warna hijau sebagai indikator bahwa elemen tersebut telah berhasil digabungkan dan diurutkan. Informasi mengenai setiap langkah, seperti proses membandingkan elemen, menyalin sisa elemen kiri atau

kanan, serta menempelkan hasil ke *array* utama, dicatat pada komponen `stepArea_2027` sehingga pengguna dapat mengikuti jalannya algoritma secara rinci.

Method `generateMergeSteps_2027()` berfungsi untuk membentuk urutan proses *Merge Sort* menggunakan teknik rekursif. Method ini membagi *array* menjadi dua bagian hingga mencapai ukuran terkecil, kemudian menyimpan setiap proses penggabungan ke dalam antrian. Dengan cara ini, program dapat menampilkan proses *Merge Sort* secara bertahap meskipun algoritma aslinya bekerja secara rekursif.

Selain itu, method `resetHighlights_2027()` digunakan untuk mengembalikan warna seluruh label ke kondisi normal sebelum langkah berikutnya dijalankan, sedangkan method `reset_2027()` berfungsi menghapus seluruh data, membersihkan tampilan visualisasi, mengosongkan area log, dan mengembalikan seluruh variabel ke kondisi awal agar pengguna dapat melakukan percobaan baru.

Pada method `main()`, program dijalankan menggunakan `SwingUtilities.invokeLater()` untuk memastikan seluruh komponen GUI dibuat pada EDT sesuai standar pemrograman Java Swing. Setelah jendela aplikasi ditampilkan, pengguna dapat memasukkan data dan menjalankan proses pengurutan langkah demi langkah.

2.7 Tugas Praktikum Pekan 8

Berikut tugas praktikum yang diberikan :

Tugas Praktikum Struktur Data

Pekan 8

Topik: Algoritma Sorting (Shell, Quick, dan Merge)

Tema: Mengurutkan Playlist Lagu

1. Deskripsi Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (A-Z) atau Durasi (terpendek ke terpanjang).

2. Aturan Penamaan (Wajib)

1. Nama Kelas: gunakan NIM lengkap. Contoh: Sorting_2411532014
2. Nama variabel & method: wajib pakai 4 digit terakhir NIM. Contoh: dataLagu_2014, shellSort_2014()
3. Jika memilih Quick Sort, method utama harus quickSort_xxxx(), jika Merge: mergeSort_xxxx(), jika Shell: shellSort_xxxx()

3. Struktur Program

Kelas Lagu (sederhana)

Atribut: judul (String), penyanyi (String), durasi (int)

Kelas Sorting_NIM

1. Minimal harus ada:
2. Array dataLagu_xxxx untuk menyimpan maksimal 20 lagu
3. Method inputData_xxxx() – untuk mengisi data awal (minimal 7 lagu)
4. Method pilihAlgoritma – cukup implementasikan SATU dari tiga di bawah:
 - a. shellSort_xxxx() – urutkan berdasarkan judul A-Z
 - b. quickSort_xxxx() – urutkan berdasarkan durasi ascending
 - c. mergeSort_xxxx() – urutkan berdasarkan judul A-Z
5. Method tampilData_xxxx() – menampilkan sebelum dan sesudah sorting

4. Ketentuan Pilihan

Pilih HANYA 1 algoritma. Tulis di laporan alasan memilih algoritma tersebut (4-5 kalimat). Tidak perlu membuat ketiganya, fokus pada implementasi yang benar dan sesuai aturan penamaan.

Gambar 2.1

5. Contoh Output

— Sorting Playlist NIM: 2411532014 —
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:

1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik

Data Setelah Quick Sort (Durasi Asc):

1. La Perla - 196 detik
2. La Rumba Del Perdon - 252 detik
3. Mio Cristo Piange Diamanti - 270 detik

6. Kriteria Penilaian

1. Algoritma berjalan benar sesuai pilihan
2. Aturan penamaan NIM 100% dipatuhi
3. Sorting bisa untuk minimal 7 data
4. Menampilkan data sebelum dan sesudah
5. Laporan menjelaskan pilihan algoritma dan kompleksitasnya

Gambar 2.2

Berikut kode program Java dari tugas praktikum :

```
public class Lagu_2511532027 {  
    private String judul_2027, penyanyi_2027;  
    private int durasi_2027;  
  
    public Lagu_2511532027(String judul_2027, String penyanyi_2027, int  
durasi_2027) {  
        this.judul_2027 = judul_2027;  
        this.penyanyi_2027 = penyanyi_2027;  
        this.durasi_2027 = durasi_2027;  
    }  
  
    public String getJudul_2027() {  
        return judul_2027;  
    }  
  
    public String getPenyanyi_2027() {  
        return penyanyi_2027;  
    }  
  
    public int getDurasi_2027() {  
        return durasi_2027;  
    }  
}
```

Kode program tersebut digunakan untuk merepresentasikan data lagu dalam bentuk sebuah kelas objek pada bahasa pemrograman Java. Kelas Lagu_2511532027 berfungsi sebagai struktur data yang menyimpan informasi mengenai sebuah lagu, yaitu judul lagu, nama penyanyi, dan durasi lagu. Dengan menggunakan pendekatan berorientasi objek, setiap lagu dapat direpresentasikan sebagai sebuah objek yang memiliki atribut dan metode tersendiri sehingga memudahkan pengelolaan data dalam program.

Pada kelas ini, terdapat tiga atribut private, yaitu judul_2027, penyanyi_2027, dan durasi_2027. Atribut judul_2027 digunakan untuk menyimpan nama atau judul lagu, atribut penyanyi_2027 digunakan untuk menyimpan nama penyanyi yang membawakan lagu tersebut, sedangkan atribut durasi_2027 digunakan untuk menyimpan lama durasi lagu dalam satuan tertentu, misalnya detik

atau menit. Penggunaan akses modifier private bertujuan untuk menjaga keamanan data agar tidak dapat diakses secara langsung dari luar kelas.

Kelas ini juga memiliki konstruktor yang digunakan untuk menginisialisasi nilai atribut saat objek dibuat. Konstruktor menerima tiga parameter, yaitu judul lagu, nama penyanyi, dan durasi lagu. Nilai dari parameter tersebut kemudian disimpan ke dalam atribut yang sesuai menggunakan kata kunci *this*, sehingga setiap objek lagu yang dibuat dapat memiliki data yang berbeda sesuai kebutuhan pengguna.

Kelas ini menyediakan tiga method getter, yaitu *getJudul_2027()*, *getPenyanyi_2027()*, dan *getDurasi_2027()*. Method-method tersebut digunakan untuk mengambil nilai dari masing-masing atribut tanpa memberikan akses langsung terhadap data yang disimpan. Dengan demikian, prinsip *encapsulation* dalam pemrograman berorientasi objek tetap terjaga karena data hanya dapat diakses melalui method yang telah disediakan.

```
import java.util.Scanner;

public class Sorting_2511532027 {

    Lagu_2511532027[] dataLagu_2027 = new Lagu_2511532027[20];
    int jumlahData_2027;

    public void inputData_2027() {
        Scanner input_2027 = new Scanner(System.in);
        do {
            System.out.print("Masukkan jumlah lagu (7-20): ");
            jumlahData_2027 = input_2027.nextInt();
            input_2027.nextLine();
            if (jumlahData_2027 < 7 || jumlahData_2027 > 20) {
                System.out.println("Jumlah lagu harus antara 7 sampai 20!");
            }
        } while (jumlahData_2027 < 7 || jumlahData_2027 > 20);
        for (int i_2027 = 0; i_2027 < jumlahData_2027; i_2027++) {
            System.out.println("\nData Lagu ke-" + (i_2027 + 1));
            System.out.print("Judul      : ");
            String judul_2027 = input_2027.nextLine();
```

```

        System.out.print("Penyanyi : ");
        String penyanyi_2027 = input_2027.nextLine();
        System.out.print("Durasi   : ");
        int durasi_2027 = input_2027.nextInt();
        input_2027.nextLine();
        dataLagu_2027[i_2027] = new Lagu_2511532027(judul_2027,
penyanyi_2027, durasi_2027);
    }
}

public void tampilData_2027() {
    for (int i_2027 = 0; i_2027 < jumlahData_2027; i_2027++) {
        System.out.println((i_2027 + 1) + ". " +
dataLagu_2027[i_2027]
                                .getJudul_2027() + " - " +
dataLagu_2027[i_2027]
                                .getPenyanyi_2027() + " - " +
dataLagu_2027[i_2027]
                                .getDurasi_2027() + " detik");
    }
}

private void tukar_2027(int i_2027, int j_2027) {
    Lagu_2511532027 sementara_2027 = dataLagu_2027[i_2027];
    dataLagu_2027[i_2027] = dataLagu_2027[j_2027];
    dataLagu_2027[j_2027] = sementara_2027;
}

public void shellSort_2027() {
    for (int gap_2027 = jumlahData_2027 / 2; gap_2027 > 0; gap_2027
/= 2) {
        for (int i_2027 = gap_2027; i_2027 < jumlahData_2027;
i_2027++) {
            Lagu_2511532027 temp_2027 = dataLagu_2027[i_2027];
            int j_2027 = i_2027;
            while (j_2027 >= gap_2027 && dataLagu_2027[j_2027 -
gap_2027].getJudul_2027()

```

```

        .compareToIgnoreCase(temp_2027.getJudul_2027()) > 0) {
            dataLagu_2027[j_2027] = dataLagu_2027[j_2027 -
gap_2027];

            j_2027 -= gap_2027;
        }
        dataLagu_2027[j_2027] = temp_2027;
    }
}

public void quickSort_2027(int kiri_2027, int kanan_2027) {
    int i_2027 = kiri_2027;
    int j_2027 = kanan_2027;
    int pivot_2027 = dataLagu_2027[(kiri_2027 + kanan_2027) /
2].getDurasi_2027();
    while (i_2027 <= j_2027) {
        while (dataLagu_2027[i_2027].getDurasi_2027() < pivot_2027)
        {
            i_2027++;
        }
        while (dataLagu_2027[j_2027].getDurasi_2027() > pivot_2027)
        {
            j_2027--;
        }
        if (i_2027 <= j_2027) {
            tukar_2027(i_2027, j_2027);
            i_2027++;
            j_2027--;
        }
    }
    if (kiri_2027 < j_2027) {
        quickSort_2027(kiri_2027, j_2027);
    }
    if (i_2027 < kanan_2027) {
        quickSort_2027(i_2027, kanan_2027);
    }
}
}

```

```

public void mergeSort_2027(int kiri_2027, int kanan_2027) {
    if (kiri_2027 < kanan_2027) {
        int tengah_2027 = (kiri_2027 + kanan_2027) / 2;
        mergeSort_2027(kiri_2027, tengah_2027);
        mergeSort_2027(tengah_2027 + 1, kanan_2027);
        gabung_2027(kiri_2027, tengah_2027, kanan_2027);
    }
}

public void gabung_2027(int kiri_2027, int tengah_2027, int
kanan_2027) {
    Lagu_2511532027[] bantu_2027 = new Lagu_2511532027[20];
    int i_2027 = kiri_2027;
    int j_2027 = tengah_2027 + 1;
    int k_2027 = kiri_2027;
    while (i_2027 <= tengah_2027 && j_2027 <= kanan_2027) {
        if (dataLagu_2027[i_2027].getJudul_2027()
.compareToIgnoreCase(dataLagu_2027[j_2027].getJudul_2027()) <= 0) {
            bantu_2027[k_2027++] = dataLagu_2027[i_2027++];
        } else {
            bantu_2027[k_2027++] = dataLagu_2027[j_2027++];
        }
    }
    while (i_2027 <= tengah_2027) {
        bantu_2027[k_2027++] = dataLagu_2027[i_2027++];
    }
    while (j_2027 <= kanan_2027) {
        bantu_2027[k_2027++] = dataLagu_2027[j_2027++];
    }
    for (int indeks_2027 = kiri_2027; indeks_2027 <= kanan_2027;
indeks_2027++) {
        dataLagu_2027[indeks_2027] = bantu_2027[indeks_2027];
    }
}

public static void main(String[] args) {

```

```

Scanner input_2027 = new Scanner(System.in);
Sorting_2511532027 playlist_2027 = new Sorting_2511532027();
playlist_2027.inputData_2027();
System.out.println("\n=== Sorting Playlist NIM: 2511532027
===");
System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
");
int pilihan_2027 = input_2027.nextInt();
System.out.println("\nData Sebelum Sorting:");
playlist_2027.tampilData_2027();
switch (pilihan_2027) {
    case 1:
        playlist_2027.shellSort_2027();
        System.out.println("\nData Setelah Shell Sort (Judul A-
Z):");
        break;
    case 2:
        playlist_2027.quickSort_2027(0,
playlist_2027.jumlahData_2027 - 1);
        System.out.println("\nData Setelah Quick Sort (Durasi
Asc):");
        break;
    case 3:
        playlist_2027.mergeSort_2027(0,
playlist_2027.jumlahData_2027 - 1);
        System.out.println("\nData Setelah Merge Sort (Judul A-
Z):");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
        return;
}
playlist_2027.tampilData_2027();
}
}

```

Kode program tersebut digunakan untuk mengelola dan mengurutkan data palylist lagu menggunakan *Shell Sort*, *Quick Sort*, *Merge Sort*. Program ini memungkinkan pengguna memasukkan data lagu yang terdiri dari judul lagu, nama

penyanyi, dan durasi lagu, kemudian memilih metode pengurutan yang ingin digunakan. Dengan adanya beberapa pilihan algoritma *sorting*, pengguna dapat mempelajari dan membandingkan cara kerja masing-masing algoritma dalam mengurutkan data objek bertipe Lagu_2511532027.

Pada kelas Sorting_2511532027, data lagu disimpan dalam *array* dataLagu_2027 yang mampu menampung hingga 20 objek lagu. Variabel jumlahData_2027 digunakan untuk menyimpan jumlah lagu yang dimasukkan pengguna. Method inputData_2027() berfungsi untuk menerima input data lagu dari pengguna melalui keyboard menggunakan objek Scanner. Program membatasi jumlah lagu yang dapat dimasukkan antara 7 hingga 20 data. Setelah jumlah lagu valid, pengguna diminta memasukkan judul, penyanyi, dan durasi untuk setiap lagu yang kemudian disimpan sebagai objek Lagu_2511532027 di dalam *array*.

Method tampilData_2027() digunakan untuk menampilkan seluruh data lagu yang tersimpan dalam *array*. Setiap data ditampilkan dalam format nomor urut, judul lagu, nama penyanyi, dan durasi lagu. Method ini dipanggil sebelum dan sesudah proses pengurutan sehingga pengguna dapat melihat perubahan urutan data setelah *sorting* dilakukan. Selain itu, terdapat method tukar_2027() yang berfungsi untuk menukar posisi dua objek lagu dalam *array* dan digunakan pada proses *Quick Sort*.

Proses pengurutan menggunakan *Shell Sort* dilakukan pada method shellSort_2027(). Algoritma ini mengurutkan data berdasarkan judul lagu secara alfabetis dari A sampai Z dengan memanfaatkan nilai *gap* yang semakin mengecil. Perbandingan dilakukan menggunakan method compareToIgnoreCase() sehingga huruf besar dan huruf kecil dianggap sama saat proses pengurutan berlangsung. Dengan metode ini, data lagu dapat tersusun berdasarkan judul secara lebih efisien dibandingkan *Insertion Sort* biasa.

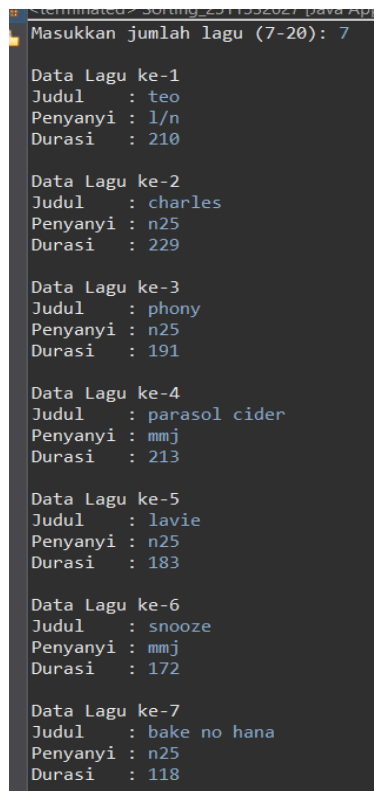
Pengurutan menggunakan *Quick Sort* dilakukan pada method quickSort_2027(). Pada algoritma ini, nilai pivot diambil dari durasi lagu yang berada di tengah *array*. Selanjutnya data lagu dibagi menjadi dua bagian berdasarkan nilai pivot, yaitu lagu yang memiliki durasi lebih kecil ditempatkan di sebelah kiri dan lagu yang memiliki durasi lebih besar ditempatkan di sebelah kanan. Proses ini dilakukan secara rekursif hingga seluruh data terurut berdasarkan

durasi lagu secara *ascending*. Pertukaran posisi data dilakukan menggunakan method `tukar_2027()`.

Pengurutan menggunakan *Merge Sort* dilakukan melalui method `mergeSort_2027()` dan `gabung_2027()`. Method `mergeSort_2027()` membagi *array* menjadi beberapa bagian yang lebih kecil secara rekursif hingga tersisa satu elemen pada setiap bagian. Setelah itu, method `gabung_2027()` bertugas menggabungkan kembali bagian-bagian tersebut dengan membandingkan judul lagu menggunakan `compareToIgnoreCase()`. Hasil akhirnya adalah data lagu yang tersusun berdasarkan judul secara alfabetis dari A sampai Z.

Pada method `main()`, program dijalankan dengan membuat objek `Sorting_2511532027`, kemudian memanggil method `inputDat_2027()` untuk menerima data dari pengguna. Setelah data dimasukkan, pengguna diberi pilihan algoritma *sorting* yang akan digunakan, yaitu *Shell Sort*, *Quick Sort*, *Merge Sort*. Program kemudian menampilkan data sebelum pengurutan, menjalankan algoritma yang dipilih, dan menampilkan hasil pengurutan setelah proses selesai.

Hasil / Output :



```
Sorting_2511532027 Java App
Masukkan jumlah lagu (7-20): 7

Data Lagu ke-1
Judul   : teo
Penyanyi : l/n
Durasi  : 210

Data Lagu ke-2
Judul   : charles
Penyanyi : n25
Durasi  : 229

Data Lagu ke-3
Judul   : phony
Penyanyi : n25
Durasi  : 191

Data Lagu ke-4
Judul   : parasol cider
Penyanyi : mmj
Durasi  : 213

Data Lagu ke-5
Judul   : lavie
Penyanyi : n25
Durasi  : 183

Data Lagu ke-6
Judul   : snooze
Penyanyi : mmj
Durasi  : 172

Data Lagu ke-7
Judul   : bake no hana
Penyanyi : n25
Durasi  : 118
```

Gambar 2.3

```
=== Sorting Playlist NIM: 2511532027 ===  
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2  
  
Data Sebelum Sorting:  
1. teo - l/n - 210 detik  
2. charles - n25 - 229 detik  
3. phony - n25 - 191 detik  
4. parasol cider - mmj - 213 detik  
5. lavie - n25 - 183 detik  
6. snooze - mmj - 172 detik  
7. bake no hana - n25 - 118 detik  
  
Data Setelah Quick Sort (Durasi Asc):  
1. bake no hana - n25 - 118 detik  
2. snooze - mmj - 172 detik  
3. lavie - n25 - 183 detik  
4. phony - n25 - 191 detik  
5. teo - l/n - 210 detik  
6. parasol cider - mmj - 213 detik  
7. charles - n25 - 229 detik
```

Gambar 2.4

BAB 3

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *sorting* seperti *Shell Sort*, *Quick Sort*, dan *Merge Sort* merupakan metode dasar pengurutan data yang memiliki cara kerja yang berbeda-beda dalam menyusun data secara *ascending*. *Shell Sort* meningkatkan efisiensi *Insertion Sort* dengan menggunakan nilai *gap* untuk mempercepat perpindahan elemen, *Quick Sort* menerapkan metode *Divide and Conquer* dengan membagi data berdasarkan pivot, dan *Merge Sort* mengurutkan data melalui proses pembagian dan penggabungan kembali secara terstruktur. Melalui praktikum ini, mahasiswa dapat memahami proses kerja masing-masing algoritma, mengimplementasikannya ke dalam program Java, serta mengetahui kelebihan dan kekurangan dari setiap metode *sorting*. Selain itu, pembuatan program berbasis GUI juga membantu memvisualisasikan proses pengurutan sehingga algoritma lebih mudah dipahami.

Dalam pelaksanaan praktikum selanjutnya, disarankan agar mahasiswa lebih banyak melakukan percobaan menggunakan jumlah data yang berbeda-beda untuk melihat perbandingan kinerja masing-masing algoritma *sorting*. Mahasiswa juga dapat mencoba mengembangkan program dengan menambahkan fitur animasi, pilihan metode *sorting*, maupun pengurutan secara *descending* agar program lebih interaktif. Selain itu, pemahaman mengenai logika algoritma dan struktur program perlu terus dilatih agar kemampuan pemrograman serta analisis terhadap efisiensi algoritma dapat meningkat dengan lebih baik.

DAFTAR PUSTAKA

- [1] <https://www.geeksforgeeks.org/shell-sort/> [Diakses 02 Juni 2026]
- [2] <https://www.geeksforgeeks.org/quick-sort/> [Diakses 02 Juni 2026]
- [3] <https://www.geeksforgeeks.org/quick-sort/> [Diakses 02 Juni 2026]

LAMPIRAN

Gambar 2.1	28
Gambar 2.2	28
Gambar 2.3	36
Gambar 2.4	37